

CSS Aural Reference

Master Audio, Speech, and Accessibility Properties for Screen Readers

What are CSS Aural Properties?

CSS Aural properties control how web content is presented through audio and speech synthesis. Originally part of CSS 2.0, they're essential for accessibility—enabling screen readers to properly render content for visually impaired users. Modern browsers implement these through voice and speech-related properties for more control over audio presentation.

Key Use Cases

- **Accessibility:** Screen reader optimization for visually impaired users
- **Audio Descriptions:** Control how content is read aloud
- **Speech Synthesis:** Customize voice output in speech-enabled browsers
- **Multi-Modal Content:** Enhance experiences beyond visual design
- **Compliance:** Meet WCAG accessibility standards

Browser Support Status

⚠️ **Important:** CSS Aural properties have limited browser implementation. Most modern browsers don't directly support these properties. Support is primarily through speech-to-text APIs and screen readers (JAWS, NVDA, VoiceOver) that read content semantically. Focus on semantic HTML and ARIA instead.

Original CSS Aural Properties (Deprecated)

The original CSS 2.0 Aural specification defined properties for controlling audio output. While largely superseded by modern APIs, understanding them is important for context and legacy systems.

Volume Property

```
/* Control loudness of audio output */
volume: silent | x-soft | soft | medium | loud | x-loud | 0-100;
```

Values Explained

silent: Muted completely (0%)
x-soft: Very quiet (25%)
soft: Quiet (50%)
medium: Normal volume (75%, default)
loud: Loud (100%)
x-loud: Maximum volume (125%)
0-100: Numeric percentage value

Pitch Property

```
/* Control voice pitch (frequency) */
pitch: x-low | low | medium | high | x-high | ;
```

Values Explained

x-low: Very deep voice
low: Lower pitch
medium: Normal pitch (default)
high: Higher pitch
x-high: Very high pitch
<frequency>: Specific frequency in Hz (e.g., 120Hz, 150Hz)

Speech Rate Property

```
/* Control how fast content is spoken */
speech-rate: x-slow | slow | medium | fast | x-fast | | ;
```

Values Explained

x-slow: Very slow (60 words/min)
slow: Slower (80 words/min)
medium: Normal (100 words/min, default)
fast: Faster (120 words/min)
x-fast: Very fast (150 words/min)
Percentage/Number: 1.0 = normal speed, 0.5 = half speed, 2.0 = double speed

Voice Family Property

```
/* Select voice characteristics */
voice-family: male | female | child | | ;
```

Voice Types

male: Male voice characteristics
female: Female voice characteristics
child: Younger-sounding voice
<name>: Specific voice (e.g., "Microsoft Zira", "Google UK English Female")
<generic-voice>: Fallback voice category

Modern Alternatives to Aural CSS

Since traditional CSS Aural properties lack browser support, modern web development uses these alternatives for audio and accessibility control.

1. Web Speech API (JavaScript)

```
/* Modern JavaScript API for speech synthesis */
const utterance = new SpeechSynthesisUtterance('Hello world');
utterance.rate = 1.0; // 0.1 to 10
utterance.pitch = 1.0; // 0 to 2
utterance.volume = 1.0; // 0 to 1
utterance.voice = voices[0]; // Select available voice
speechSynthesis.speak(utterance);
```

✅ **Recommended:** Use Web Speech API for speech synthesis control. It provides cross-browser compatibility and fine-grained control over voice output.

2. ARIA Attributes (Accessibility)

```
<!-- Tell screen readers how to announce content -->
<div role="alert" aria-live="polite" aria-label="Custom label">
  Important notification
</div>

<!-- Screen readers use semantic meaning -->
<article>Main content</article>
<nav>Navigation</nav>
<aside>Sidebar</aside>
```

✅ **Best Practice:** ARIA and semantic HTML are the primary way to control screen reader behavior. This is the modern standard for accessibility.

3. Semantic HTML

```
<!-- Screen readers understand semantic meaning -->
<h1>Main heading</h1> <!-- Clearly marked as top-level -->
<h2>Section heading</h2> <!-- Hierarchy preserved -->
<button>Click me</button> <!-- Announced as interactive -->
<img alt="Description" > <!-- Alt text essential for images -->
<form><label for="input">Name</label><input id="input"></form>
```

Web Speech API - Modern Speech Synthesis

The Web Speech API provides JavaScript control over speech synthesis with properties for rate, pitch, and volume—replacing deprecated CSS aural properties.

SpeechSynthesisUtterance Properties

Property	Range	Default	Purpose
text	String	""	Text to be spoken
lang	Language code	en-US	Language for speech (e.g., fr-FR, es-ES)
voice	SpeechSynthesisVoice	Default system	Voice to use for synthesis
volume	0 to 1	1	Loudness (0 = mute, 1 = maximum)
rate	0.1 to 10	1	Speaking speed (0.5 = half speed, 2 = double)
pitch	0 to 2	1	Voice pitch (0 = deepest, 1 = normal, 2 = highest)

Example: Complete Speech Synthesis

```
// Get available voices
const voices = speechSynthesis.getVoices();

// Create utterance
const utterance = new SpeechSynthesisUtterance(
  'Hello! This is a test message.'
);

// Configure speech
utterance.voice = voices[0]; // Select first available voice
utterance.lang = 'en-US'; // Language
utterance.rate = 1.0; // Normal speed
utterance.pitch = 1.0; // Normal pitch
utterance.volume = 1.0; // Full volume

// Event listeners
utterance.onstart = () => console.log('Speaking started');
utterance.onend = () => console.log('Speaking ended');
utterance.onerror = (event) => console.error('Error:', event);

// Speak
speechSynthesis.speak(utterance);
```

Optimizing for Screen Readers

While CSS aural properties aren't directly supported, screen readers honor semantic HTML and ARIA attributes. Focus on these for proper audio presentation.

Key Screen Reader Techniques

Hide from Screen Readers

aria-hidden="true"

Hide decorative elements that shouldn't be announced. Use sparingly for icons, spacers, etc.

Custom Labels

aria-label="Description"

Provide custom text for screen readers when visual text is insufficient.

Link Descriptions

aria-describedby="id"

Link generic text (like "read more") to detailed descriptions.

Live Regions

aria-live="polite|assertive"

polite: Announce updates when user pauses
assertive: Announce immediately (interrupting)

Landmark Regions

role="main|nav|complementary"

Define page regions so screen reader users can navigate quickly.

Headings Hierarchy

<h1>...<h6>

Use semantic headings. Screen readers skip to specific levels for navigation.

HTML Best Practices

```
<!-- Use semantic HTML -->
<header>...</header><!-- Page header */
<nav>...</nav> <!-- Navigation */
<main>...</main> <!-- Main content */
<aside>...</aside> <!-- Sidebar/related */
<footer>...</footer> <!-- Page footer */

<!-- Use form labels -->
<label for="email">Email</label>
<input id="email" type="email">

<!-- Provide alt text for images -->


<!-- Use buttons for actions -->
<button onClick="doSomething()">Click me</button>
```

Essential ARIA Properties for Audio Control

ARIA (Accessible Rich Internet Applications) attributes tell screen readers how to announce dynamic content.

Attribute	Values	Purpose	Example
aria-label	Text string	Provide accessible name	aria-label="Close menu"
aria-live	off, polite, assertive	Announce updates dynamically	aria-live="polite"
aria-hidden	true, false	Hide from screen readers	aria-hidden="true"
aria-describedby	ID reference	Link to detailed description	aria-describedby="help-text"
role	main, nav, search, etc.	Define semantic role	role="main"
aria-atomic	true, false	Read entire region when updated	aria-atomic="true"

Practical Examples

Example 1: Accessible Form

```
<form aria-label="Login form">
  <label for="username">Username</label>
  <input id="username" type="text" required>

  <label for="password">Password</label>
  <input id="password" type="password" required>
  <span id="password-hint">At least 8 characters</span>
  <input aria-describedby="password-hint">

  <button type="submit">Log In</button>
</form>
```

Example 2: Live Region for Updates

```
<!-- Announce new messages -->
<div aria-live="polite" aria-atomic="true" id="messages">
  <!-- Updated dynamically by JavaScript -->
</div>

<!-- JavaScript updates content -->
document.getElementById('messages').textContent =
  'New message from Alice';
<!-- Screen reader announces immediately -->
```

Example 3: Speech Synthesis with Controls

```
<div id="speech-demo">
  <button id="speak-btn">Speak Text</button>
  <button id="pause-btn">Pause</button>
  <button id="resume-btn">Resume</button>
  <button id="stop-btn">Stop</button>

  <p id="text-to-speak">
    This is the text that will be spoken aloud.
  </p>
</div>

<script>
const utterance = new SpeechSynthesisUtterance(
  document.getElementById('text-to-speak').textContent
);
utterance.rate = 1.0;
utterance.pitch = 1.0;

document.getElementById('speak-btn').addEventListener(
  'click', () => speechSynthesis.speak(utterance));
document.getElementById('pause-btn').addEventListener(
  'click', () => speechSynthesis.pause());
document.getElementById('resume-btn').addEventListener(
  'click', () => speechSynthesis.resume());
document.getElementById('stop-btn').addEventListener(
  'click', () => speechSynthesis.cancel());
</script>
```

Best Practices & Accessibility Guidelines

🔗 **Tip 1: Semantic HTML First**
Use proper HTML tags (<button>, <form>, <h1>). This is the foundation for accessibility.

🔗 **Tip 2: Always Provide Alt Text**
 Essential for images. Alt text is read by screen readers.

🔗 **Tip 3: Label Form Inputs**
<label for="id">Label</label><input id="id"> Always associate labels with inputs.

🔗 **Tip 4: Use ARIA Sparingly**
ARIA is for when HTML doesn't provide the needed semantics. Don't override semantic HTML with incorrect ARIA roles.

🔗 **Tip 5: Test with Screen Readers**
Use NVDA (Windows), JAWS, or VoiceOver (Mac) to test your content. Real screen reader testing catches issues.

🔗 **Tip 6: Don't Rely on Color Alone**
Combine color with text, icons, or patterns. Screen reader users can't see colors.

🔗 **Tip 7: Keyboard Navigation**
All interactive elements should work with Tab, Enter, Escape. Some users can't use a mouse.

🔗 **Tip 8: Live Regions for Dynamic Content**
Use aria-live="polite" for content that updates without page reload (notifications, scores, etc.).

WCAG Accessibility Standards

Web Content Accessibility Guidelines (WCAG) define standards for accessible web content. Focus on these levels to ensure proper audio/accessibility implementation.

WCAG Level	Requirement	Impact
A (Minimum)	Basic accessibility	Semantic HTML, alt text, basic color contrast
AA (Recommended)	Enhanced accessibility	Better contrast, keyboard navigation, form labels
AAA (Best)	Advanced accessibility	High contrast, detailed descriptions, sign language video

💡 **Recommendation:** Aim for WCAG AA compliance as a baseline. This provides good accessibility for most users.

Browser & Screen Reader Support

CSS Aural Properties: Minimal to no browser support. Not recommended for production use.

Web Speech API: Chrome 25+, Edge 79+, Safari 14.1+, Firefox (limited)

ARIA Attributes: All modern browsers via screen readers (JAWS, NVDA, VoiceOver)

Semantic HTML: Universal support across all browsers and screen readers

Additional Resources

Screen Readers (Test Tools):

- NVDA (Windows) - Free, open-source
- JAWS (Windows) - Professional, feature-rich
- VoiceOver (Mac/iOS) - Built-in Apple screen reader
- TalkBack (Android) - Built-in Android screen reader

Standards & Guidelines:

- WCAG 2.1 (Web Content Accessibility Guidelines)
- WAI-ARIA (Web Accessibility Initiative - Accessible Rich Internet Applications)
- Section 508 (US Federal accessibility requirements)